

PLC Ladder Logic Trainer

Demo 9 of 17 · Part four, Example projects · 15 minutes for a first pass across all four stages; a full 45-minute period using the lesson plan below

You need: `index.html` open in any modern browser, on a projector or on each student's device. No network, no install, no accounts. Nothing is fetched at runtime, so it works with the wi-fi off.

Learning goals

One per stage, in the learner's own words — these are the four Takeaway cards on screen, verbatim. If a student can say these four things and point at the evidence for each, the demo has done its job.

1. A PLC scan reads all inputs, solves the ladder from that one snapshot, then writes all outputs — a button press does nothing until the next input read.
2. A rung sees what an earlier rung wrote this same scan, but only sees a later rung's write on the following scan — rung order is part of a ladder program's meaning, not just its layout.
3. A program is "correct" when it produces the right outputs at every step of a real test sequence — not when it merely resembles the spec.
4. The intersection's misbehavior is evidence about which instruction changed — the ladder confirms a diagnosis, it does not have to be where you start.

Where this sits on the path

An AI-built classroom tool from Part four (Applied Engineering · PLC control). Its goal is to make a real industrial control loop watchable one scan at a time, its engine is a live, genuine scan-cycle simulator (not an animation), and its assumptions — what is a deliberate teaching simplification and what would carry over to a real controller — are one tap away in **Details**. It does not depend on any earlier demo; the habit it shares with the rest of Part four is exposing exactly which numbers are computed live, which are this build's own reading of a documented standard, and which are a hand-authored fixture.

One question for the session

How does a controller that never stops looping actually think — and what does that loop cost you if you get its order wrong?

"This is not a wiring diagram and not an animation. It is a real, deterministic scan-cycle engine: read every input, solve every rung in order, write every output, forever. Everything we do for the next fifteen minutes is about what that one loop implies — for timing, for the order you write rungs in, for what 'correct' means, and for how you'd diagnose it when something breaks."

Stage 1 · Trainer — how does one scan run?

Question: In what order does the PLC actually do its work, and when does a button press take effect?

Predict prompt: Pause the PLC, then press START. Will the ladder show it as ON before or after you press **Step Scan**? Take a show of hands before anyone steps.

Activity (5 minutes): 0:00 load Program 1 (Basic Timed Stoplight), running · 1:00 press **II Pause**, open the watch window · 1:30 press START, then **Step Scan** once and read both image tables · 3:00 step a few more scans and narrate what changed and when · 4:00 open **Details** for the formal scan order.

Expected observation: The input image table shows START as unchanged until the next Step Scan, at which point it flips — the ladder's power flow and the watch window's flashed row both update on that same step, not before.

Check-yourself answer key: "Not until the next Step Scan (or the next running scan)." The tempting wrong answer treats the PLC as event-driven ("immediately, the instant you press it") — the paused state is exactly what refutes it: nothing moves until you advance the scan yourself.

Teaching limit: Do not let "the PLC reads inputs" become "the PLC reacts instantly." Stage 2 depends on the class already treating "scan" as a discrete, ordered event, not a continuous flow.

Misconception: outputs change mid-rung. Real outputs update only at the end of the scan, all together, from the output image table built while solving. Watch the output image table while stepping: it holds its old values until the whole rung list has been solved once.

Misconception: pressing a button acts instantly. It acts at the next input scan. In step mode a press does nothing until you step — this is the whole point of the Predict prompt above.

Misconception: a timer counts wall-clock time. A TON accumulates only while its rung is true, in simulated scan-time increments, and resets the moment its rung goes false — it is not reading a real clock, which is also what makes every run bit-for-bit reproducible regardless of speed.

Stage 2 · Editor — why does rung order matter?

Question: Does it matter whether a rung comes before or after the one it depends on?

Predict prompt: Load Program 6A, then 6B (same two rungs, opposite order). Which variant makes B_ECHO react one scan later than B_SRC?

Activity (8 minutes): 0:00 load 6A, hold START, single-step and watch B_SRC then B_ECHO · 2:00 load 6B, repeat · 4:00 in the Editor, place a rung from the palette and press **Run My Program** · 6:00 discuss why the same two instructions in opposite order are two different programs.

Expected observation: In 6A, B_SRC turns on the scan you press START, but B_ECHO lags exactly one scan behind it. In 6B, both update the same scan.

Check-yourself answer key: "Yes — the same logic, one scan later, is a different program." The wrong option treats ladder like relay wiring, where physical order genuinely would not matter; a PLC scan is not continuous current flow, it is a fixed, repeated solve order.

Teaching limit: This is about scan-order semantics, not style. The editor's constrained palette (nine instructions, no nested branches) is a teaching simplification — see Model and boundaries.

Misconception: ladder is a schematic. It looks like relay wiring but is a **program**: solved in order, one rung at a time, over and over. Electricity flows everywhere at once in a real relay panel; a PLC scan does not.

Stage 3 · Challenges — can you build it?

Question: Can you turn a plain-English requirement into a program that survives a real test, not just one that looks right?

Predict prompt: Before pressing Check on the seal-in challenge — will a seal-in built with a single OTE survive a momentary loss of the START signal, or drop out?

Activity (10 minutes): 0:00 read the seal-in spec and required tags · 1:00 build in the palette · 3:00 press **Check my program**, read the first failing step if any · 5:00 fix and recheck; fast finishers move to Delayed Start, then the full stoplight-from-spec challenge.

Expected observation: A PASSED banner names all steps the scripted sequence checked; a NOT YET banner names the first step that failed and when, not merely "wrong."

Check-yourself answer key: "That your program's outputs matched the scripted test at every step it checked." A different, differently-wired program that behaves the same on that sequence passes equally well — there is no single intended wiring the grader is comparing against.

Teaching limit: The four challenges check one behavioral spec each; they are not a style or efficiency grade, and passing one is not a certificate for real PLC work.

Misconception: a PASSED badge means my program is the intended, correct solution. It means your program matched the scripted test's expected outputs at every step the grader checked. Two structurally different programs — different tags, different instruction choices, even a different number of rungs — that produce the same outputs on that sequence both pass. Ask a fast finisher to compare their wiring with a neighbor's passing solution to make this concrete.

Stage 4 · Troubleshoot — can you find the fault?

Question: Can you find an injected fault from how the intersection misbehaves, before you look at the ladder?

Predict prompt: Hide the ladder pane (the Predict card's own toggle), inject a fault, and guess which instruction changed from the intersection's behavior alone — then reveal to check.

Activity (12 minutes): 0:00 check "Hide ladder pane" · 1:00 press **Inject random fault**, watch the intersection for a full cycle or two · 3:00 write down a guess · 4:00 press **I've diagnosed it — reveal** and compare · repeat 2–3 times, using the teacher <details> to pick a specific fault for a targeted round (e.g. the double-coil fault for the Check-yourself item below).

Expected observation: The reveal card names the fault, the symptom, and the explanation, and highlights the exact patched rungs on the ladder.

Check-yourself answer key: "The one solved last, every single scan." The wrong option ("whichever turned on first") is the natural but incorrect guess — turning on first offers no protection, because every OTE on the rungs still executes in program order every scan.

Teaching limit: This engine's own last-write-wins behavior is verified by its self-test; it is a close, well-corroborated reading of how a single real controller resolves a duplicate coil in one scan, not a certified statement about every configuration — see Details.

Misconception: two coils with the same tag both work. The **last one solved wins** every scan (see the double-coil fault) — the earlier coil's write is simply overwritten before the scan ends. It does not error, and it does not average; it is silently overwritten, which is exactly why it is worth a dedicated fault.

Model and boundaries

input scan → solve rungs 0..N-1 (branches top-to-bottom) → output scan

- Every input is latched into an input image table once per scan; the ladder is solved strictly top-to-bottom, left-to-right, from that one snapshot; every output is written from the output image table at once, at the end of the scan.
- Timers and counters accumulate in **simulated** scan time (Timing & speed's Scan-time setting), never the wall clock — a run is bit-for-bit reproducible at any Speed multiplier.
- Series instructions = AND, parallel branches = OR — this engine's own reading of Allen-Bradley/Logix-style rung-condition-in/rung-condition-out execution, **Reasoned** rather than **Sourced** because the primary Rockwell documentation could not be read directly in this research pass (see Sources). Corroborated by independent secondary references for a single controller, single scan, non-retentive OTE — exactly what this engine models.
- This engine's duplicate-OTE "last write wins" behavior is **Live** — verified by reading `src/engine.js` and by one of the 21 assertions in Settings → Run engine self-test. Whether every real Allen-Bradley/Logix controller behaves identically in every configuration (retentive instructions, distributed I/O) is **Reasoned**, with that boundary stated on screen.
- The editor's palette is a deliberately constrained subset (nine instruction types, no nested branches, a fixed tag list) — a teaching simplification, not a limit of real ladder logic.

This is a teaching engine, not a certified PLC and not IEC 61131-3 conformance-tested. It models one deterministic reading of Allen-Bradley-style ladder execution for the classroom, not any vendor's certified runtime or firmware version. Full per-stage assumption lists are in the app's **Details** drawer, which is the surface students should be sent to when they ask "but is that a real PLC?".

One deliberate simplification, worth a five-minute extension. These programs step straight from a yellow to the cross street's green. A real signal inserts an *all-red clearance interval* first (MUTCD §4D.26 governs the yellow-change and red-clearance intervals) so the intersection empties before conflicting movements get a green. It is left out to keep the sequence short enough to read on one screen — ask the class to add it in the Editor, and watch them discover it costs a timer and a step in every phase.

Discussion questions

- Why can't both directions ever be green in Program 1, even if a timer misbehaves? (Two independent defenses: the step sequence *and* the XIO interlock contacts — Details names both.)
- Why is the walk request latched (ONS + OTL) instead of wired straight to the walk sign? What would happen if it weren't?
- In the Scan Order Demo, which variant (6A or 6B) would you choose for an emergency-stop rung, and why?
- Seal-in vs. OTL for the motor: which restarts by itself after a power blip? Which is safer, and when?
- Our simulated scan defaults to 20 ms. What real-world inputs could a PLC miss entirely between scans, and how would you catch them?
- The page labels some claims Live, some Reasoned. Find one of each in Stage 4's Details and say what each label is promising you — and NOT promising you.

Presenter controls

The four stage tabs run across the top and are never gated — a class can jump straight to Troubleshoot. **Details** in each stage intro opens that stage's mechanism, assumptions and sources. ? reopens the Guide. ⚙ Settings offers **Open Presenter Notes** (this document), **Presentation mode**, **Reset**, and this demo's own **engine self-test** (21 live assertions about scan semantics).

Presentation mode scales the page up one notch and adds a **Notes** button to the header; it also scales the ladder/sim/editor/challenge panels at their own larger factor (this demo's own mechanism, unrelated to the page-text scaling). **Reset** returns the demo to its opening state *without reloading the page* — Program 1 fresh, the editor emptied, all challenge progress cleared, any fault abandoned, Timing back to 1×/20 ms — and *leaves presentation mode as you set it*. The engine-run controls (Program, Run/Pause, Step Scan, Reset PLC) are visible on Trainer and Troubleshoot, where operating the PLC's timing is the point; on Editor and Challenges they are one tab away, and the live intersection keeps running underneath regardless of which stage is showing.

Keep the live engine and the teaching frame distinct. The scan-cycle mechanics, the seven programs, the ladder editor, the four challenges and the eleven faults are all unchanged by this pass — every one is **Live**, computed by the same engine, verified by its own self-test and node test suites. Only the surrounding frame (stage navigation, Predict/Try/Takeaway, Details, Check yourself) is new.

Sources and attribution

- [Rung-Condition Behavior in Studio 5000 Ladder Logic](#) (fetched 2026-09-16) — the rung-condition-in/rung-condition-out chain this build's series=AND, parallel=OR reading follows.
- [OTE \(Output Energize\) — Definition & Explanation](#) (fetched 2026-09-16) — OTE re-solves its tag from the rung condition every scan.
- Independent PLC-programming forum and reference discussions of "last rung wins" for a duplicate output coil (fetched 2026-09-16) — consistent for a single controller, single scan, non-retentive OTE (this engine's configuration); one discussion found also disputes the flat "last write always wins" framing for other hardware and configurations (retentive instructions, distributed I/O), which is why this demo's own claim is labeled Reasoned with a stated boundary rather than flatly Sourced. Rockwell's own primary instruction-set page and PDF programming manual were attempted first and could not be read directly in this research pass (a region/cookie interstitial with no instruction text, and a PDF whose text layer did not decode) — disclosed rather than silently worked around.
- MUTCD §4D.26 (yellow-change and red-clearance intervals) — spot-checked via a California MUTCD companion document confirming the section number, not a full read of the standard's text.

These sources support the underlying PLC-programming and traffic-engineering background; they do not endorse this activity. A Murray State project record supports plausible course fit; it is not named here, and the demo is not presented as any instructor's instrument or assignment.

